INFORME TÉCNICO · UNIDAD 2

API REST para la gestión de tareas

Persistencia, sesiones y notificaciones en tiempo real

Autor: Wilmer Andrés Trujillo Vera
Repositorio: https://github.com/wtrujillov1/API

1. Propósito y organización

Se desarrolló una API REST para que cada usuario pueda registrar, consultar, actualizar y eliminar sus tareas. El proyecto utiliza Node.js 24, Express 5 y SQLite. Incluye una interfaz web en español para demostrar las operaciones y observar los cambios desde dos pestañas.

El código se divide en rutas HTTP, validación, autenticación y acceso a datos. Las funciones que normalizan y comprueban las entradas se mantienen separadas de las consultas a la base de datos. Esta organización facilita revisar cada responsabilidad y evita mezclar las reglas de una tarea con la forma de almacenarla.

2. Operaciones y validación

Operación	Método y ruta	Respuesta
Registro e inicio de sesión	`POST /api/auth/register` `POST /api/auth/login`	201 / 200
Consultar y cerrar sesión	`GET /api/auth/me` `POST /api/auth/logout`	200 / 204
Listar y consultar tareas	`GET /api/tasks` `GET /api/tasks/:id`	200
Crear una tarea	`POST /api/tasks`	201
Modificar una tarea	`PATCH /api/tasks/:id`	200
Eliminar una tarea	`DELETE /api/tasks/:id?version=N`	204
Recibir notificaciones	`GET /api/events`	Flujo SSE

Cada tarea tiene un identificador, título, descripción, estado, versión y fechas de creación y actualización. El título admite entre 1 y 120 caracteres y la descripción, hasta 2000. Los estados permitidos son `pending`, `in_progress` y `completed`. La entrada se valida antes de guardar los datos.

Las respuestas con contenido siguen el formato `{"data": ...}` y los errores, `{"error": {"code": ..., "message": ...}}`. Una entrada inválida recibe **400**; una petición sin sesión, **401**; un recurso inexistente o de otro usuario, **404**; y un conflicto de versión, **409**. Las eliminaciones correctas responden **204**, sin cuerpo.

3. Persistencia y sesiones

SQLite conserva usuarios, tareas y sesiones en disco. Las consultas utilizan parámetros y restringen el acceso por propietario. Las contraseñas se almacenan mediante una derivación con `scrypt` y una sal aleatoria. La cookie de sesión es `HttpOnly` y `SameSite=Strict`; en producción también utiliza `Secure`. El identificador de sesión se renueva al iniciar sesión y se invalida al salir.

4. Conflictos y notificaciones en tiempo real

Para evitar que una edición sobrescriba otra, las actualizaciones y eliminaciones exigen la versión conocida de la tarea. La escritura comprueba esa versión en la consulta SQL. Si ya cambió, devuelve **409** y el cliente debe consultar el estado vigente antes de reintentar.

La interfaz abre una conexión autenticada con `EventSource`. Después de guardar un cambio, el servidor emite `task.created`, `task.updated` o `task.deleted`. El aviso llega únicamente a las conexiones del usuario propietario y la interfaz vuelve a consultar su listado. Se eligió Server-Sent Events (SSE) porque se necesitan avisos del servidor al navegador; las acciones del usuario siguen realizándose mediante peticiones REST.

5. Pruebas y resultados

Las **13 pruebas de integración fueron aprobadas**, sin fallos ni omisiones. Se comprobaron el CRUD, las entradas inválidas, el registro y el cierre de sesión, el aislamiento entre usuarios, la persistencia después de reiniciar, las ediciones simultáneas y la recepción de eventos. También se verificó que una sesión caducada no permita consultar datos ni recibir nuevas notificaciones.

En Chromium se comprobó el flujo de registro, acceso y gestión de tareas. Dos pestañas reflejaron las creaciones, modificaciones y eliminaciones mediante SSE. La vista móvil de 390 píxeles no presentó desbordamiento horizontal ni excepciones de JavaScript. Las capturas y el registro de verificación están en `docs/evidencias/`.

Ejemplo de uso. Una petición `POST /api/tasks` con `{"title":"Preparar informe"}` devuelve **201** y una tarea con estado `pending` y versión `1`. Al enviar `PATCH /api/tasks/:id` con `{"status":"completed","version":1}`, la respuesta es **200** y la versión pasa a `2`. Repetir la actualización con la versión anterior devuelve **409**, sin modificar la tarea. El README contiene ejemplos completos de petición y respuesta con `curl` y cookies de sesión.

6. Ejecución y alcance

El proyecto requiere Node.js 24 o superior. Desde la raíz del repositorio se ejecutan `npm ci`, `npm test` y `npm start`. El servidor utiliza por defecto el puerto 3000 y sirve la interfaz en `/`. La base de datos se crea al iniciar y las pruebas usan archivos temporales independientes de los datos de desarrollo.

La solución está preparada para un proceso Node.js y un archivo SQLite. Un despliegue con varias réplicas necesitaría compartir las notificaciones entre procesos. Para ofrecer acceso público también se requiere HTTPS, almacenamiento persistente y una gestión segura de la clave de sesión. El código y las evidencias están publicados en el repositorio indicado; la demostración se ha validado en ejecución local.

Referencias

Brown, E. (2020). *Web development with Node & Express* (2.ª ed.). O'Reilly Media.

Duckett, J. (2014). *JavaScript & jQuery: Interactive front-end web development*. Wiley.

Robbins, J. N. (2018). *Learning web design* (5.ª ed.). O'Reilly Media.